7 key principles

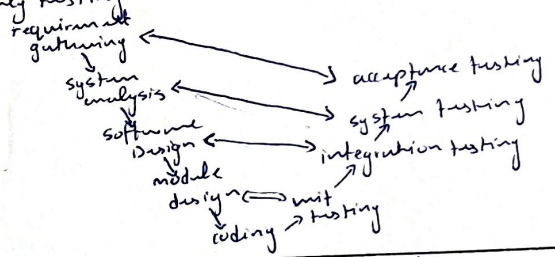
1- Rigor and Formality precise systematic rules set for Development
2- Separation of concerns tackle one aspect at a time. Separation of responsibilities might lose concerns that can only be tackled together
3- modularity split into cohesive losely coupled units. decomposition + composition.
4- abstraction hide irrelevant details. expose essential properties.
5- anticipation of change Design for maintainability
6- Generality maximize reusability
7- Incrementality Deliver in steps. allows early feed back.

Coupling independence between modules.
Want: low coupling - modules are independent.
Cohesion: module does one thing well. relatedness of elements within a module.
Want: high cohesion - module does one thing well

waterfall model (linear) Pros easy to implement
cons requirements cannot change.
-> focuses on documentation -> late testing.
-> used for stable well known requirements.

Req analysis -> Design -> planning -> coding -> testing -> Deployment & maintenance.

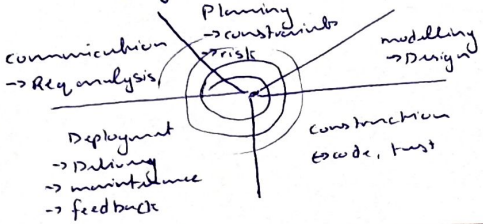
waterfall (v-model) same use cases, pros, cons as classic waterfall. used where rigorous testing is required.
-> early testing



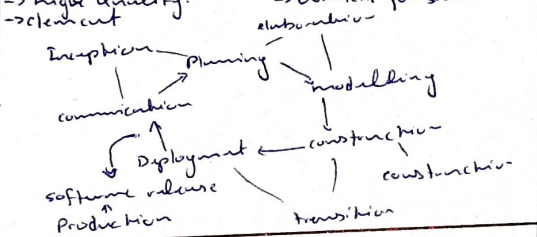
Incremental vs iterative vs prototype.

	Incremental	Iterative	Prototype
each cycle output	adds new feature	refines some feature	explores requirement
	Production ready subset/module	improved version	may be discarded
goals	grow System	Improve Quality	clarify requirement.

spiral model use when requirements are unclear and likely to change. need flexibility and control.
Pros risk management
combines prototype, iterative and waterfall. consumer review
cons complex to manage
not suitable for small projects
~~complex to manage~~ need to make sure spiral doesn't keep going on.

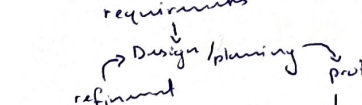


unified process model: combines best of each Design.
-> early risk reduction
-> high Quality.
-> client cut
-> complex / heavy
-> overkill for small.



Incremental several versions
-> reduced risk
-> early working product
-> requires good planning
-> integration tricky
-> not ideal if no modules.

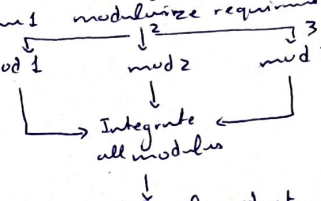
Prototyping use when requirements are unclear / ambiguous.
-> reduces costly changes later
-> user satisfaction
-> extra time and cost if not controlled.
-> poor prototypes -> poor architecture.



-> temporary model to understand requirements.

RAD Rapid Application Development
use when highly skilled team available.
-> fast Delivery
-> code reuse
-> user involvement
-> needs modularizable system
-> 60-90 Days.

Elicit Requirements



Business modeling -> Data modeling
-> ~~process~~ process modeling -> app generation
-> testing.

Requirements Engineering.
Activities
Discovery - identify stakeholders, find what they need.
Analysis - elaboration / modeling, give structure.
Agreement - negotiation, sign off, SRS doc
control - change access management, versioning.

Elicitation techniques
Direct Interviews, questionnaires, surveys, can be groups.
indirect Brainstorming, focus groups.
observation Ethnography
iterative prototyping. Scenarios, use cases.

Agile manifesto (foundation of Scrum, individuals and interactions kanban, XP) over processes and tools. working software over documentation. customer feedback valued over planning.

SCRUM continuous delivery, collaboration and adaptation
sprints (1-4 weeks) - produces increment

Product owner - backlog
Scrum master - removes blockers
Development team - self organizing.
roles

Product Backlog - all desired work
Sprint Backlog - items for current sprint
Increment - done items
artifacts

Sprint planning - what + how
Daily Scrum - DiD / will / blockers
Sprint review - Demo to stakeholders
Retrospective - how to improve.

XP (extreme programming)
PP -> pair programming finds, other reviews.
TDD -> write test first, then code to pass.

kanban workflow smooth from start to finish.
visualize all work on board
limit WIP (work in progress) fixed max number of items.
manage flow identify where work gets stuck, address it
explicit policies remove ambiguity
feedback loops regular review meeting
evolve experimentally - Improve gradually
Board has 3 columns
To Do | In Progress | Done |

testable requirements
specific, measurable, quantified
non-testable vague adjectives
"fast"
non test to testable
1- Quantify. replace words with measurables
2- Specific names of entities
3- consistent names
specify conditions
specify tests method.

Functional
what system does e.g login + sign up
non-functional
how well system does it
e.g performance, usability